

FridgeChef: AI-Powered Recipe Generator

Author: Triet Le

Course: NLP / AI 2026

1. Project Overview

FridgeChef is an AI-powered application designed to reduce food waste by generating personalized recipes based on ingredients currently available in a user's refrigerator [cite: 6, 9]. The system utilizes Natural Language Processing (NLP) for ingredient parsing and recipe generation, with an optional Computer Vision component for automated ingredient detection [cite: 7].

2. System Architecture

The application follows a modern full-stack architecture:

- **Frontend:** React (Web) or Flutter (Mobile) for user authentication and ingredient input [cite: 15, 16].
- **Backend:** FastAPI (Python) providing a REST API for communication and logic [cite: 18, 19, 45].
- **Database:** PostgreSQL for storing user profiles, dietary preferences, and saved recipes [cite: 26, 27, 46].
- **AI Integration:** OpenAI GPT API for structured recipe generation and optional computer vision for detection [cite: 23, 24, 47].

3. Implementation Details

3.1 Database Models (SQLAlchemy)

```
from sqlalchemy import Column, Integer, String, Text, ForeignKey, JSON
from sqlalchemy.orm import relationship
from .database import Base
```

```
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    email = Column(String, unique=True, index=True)
    hashed_password = Column(String)
    dietary_preferences = Column(JSON, nullable=True)
    recipes = relationship("SavedRecipe", back_populates="owner")
```

```
class SavedRecipe(Base):
    __tablename__ = "saved_recipes"
```

```

id = Column(Integer, primary_key=True, index=True)
title = Column(String)
ingredients_used = Column(JSON)
instructions = Column(Text)
user_id = Column(Integer, ForeignKey("users.id"))
owner = relationship("User", back_populates="recipes")

```

3.2 AI Service & Prompt Engineering

To mitigate the risk of hallucinated recipes, the system uses structured JSON output constraints [cite: 74, 75].

```

async def generate_recipe_logic(ingredients, preference):
    response = await openai.ChatCompletion.acreate(
        model="gpt-4-turbo-preview",
        messages=[
            {"role": "system", "content": "Return ONLY a JSON object with 'title', 'ingredients', and 'steps'. Ensure dietary compliance."},
            {"role": "user", "content": f"Generate a {preference} recipe for: {ingredients}"},
        ],
        response_format={"type": "json_object"}
    )
    return response

```

3.3 Security (JWT Authentication)

The system implements JWT-based authentication to fulfill non-functional security requirements [cite: 38].

```

from jose import jwt
from datetime import datetime, timedelta

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=30)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm="HS256")

```

4. Testing & Deployment

- **Unit Testing:** Pytest is used to verify dietary filter compliance and response structures [cite: 59].
- **Load Testing:** Targeted response time is under 3 seconds with a 95% uptime goal [cite: 37, 41, 60].

- **Deployment:** Containerized using Docker and deployed to AWS/GCP [cite: 39, 48, 61].

5. Risks & Mitigations

Risk	Mitigation Strategy
Inaccurate ingredient detection [cite: 70]	Allow manual correction via UI chips before generation [cite: 71].
API cost scaling [cite: 72]	Implement caching strategies for common ingredient queries [cite: 73].
Hallucinated recipes [cite: 74]	Use structured prompt constraints and validation layers [cite: 75].